

RAPPORT D'ANALYSE MONGODB

COLLECTION VENTES

BASE DE DONNÉES : IMMO

RÉALISÉ PAR

Boufalous Nassim

Yanis Helali

DATE

28/08/2026

RÉFÉRENTIEL GITHUB

https://github.com/Yanis389/NoSQL_M1

SOMMAIRE

PARTIE	CONTENU
I	CONTEXTE, OBJECTIFS ET ENVIRONNEMENT
II	MÉTHODOLOGIE ET INDICATEURS D'EXÉCUTION
III	COMMUNES AYANT AU MOINS 10 VENTES
IV	VENTES À MONTPELLIER ET INDEX COMPOSÉ
V	RECHERCHE GÉOGRAPHIQUE AVEC \$GEONEAR
VI	ANALYSE COMPARATIVE DES PLANS
VII	RECOMMANDATIONS, LIMITES ET CONCLUSION

Ce rapport exploite les résultats fournis pour la collection *immo.ventes*. Les requêtes analysées associent agrégation, index composé, tri et index géospatial afin d'évaluer la qualité des plans d'exécution MongoDB.

I. CONTEXTE, OBJECTIFS ET ENVIRONNEMENT

Ce rapport analyse la collection *ventes* de la base *immo*. Les documents de cette collection décrivent des ventes immobilières dont l'exploitation peut nécessiter une analyse territoriale, financière ou géographique. L'enjeu est de produire des requêtes fiables et de vérifier qu'elles utilisent les bons index.

Trois besoins sont étudiés. Le premier consiste à sélectionner les communes ayant une activité significative. Le second cible les ventes de Montpellier, triées par valeur foncière décroissante. Le troisième utilise une recherche de proximité autour des coordonnées [3.8772, 43.6119].

OBJECTIF	MESURE OU RÉSULTAT
Mesurer l'activité territoriale	266 communes disposent d'au moins 10 ventes
Consulter une commune précise	5 588 ventes de Montpellier retournées
Analyser une zone géographique	6 150 ventes à moins de 5 km
Vérifier les index	Index composé et index 2dsphere utilisés

Les opérations utilisées sont *aggregate()*, *find()*, *sort()*, *\$geoNear* et *explain("executionStats")*. Cette dernière commande permet de distinguer une requête fonctionnelle d'une requête réellement optimisée.

II. MÉTHODOLOGIE ET INDICATEURS D'EXÉCUTION

Les requêtes ont été analysées avec une approche en deux temps : obtenir le résultat métier, puis observer le plan d'exécution. Le plan permet notamment de voir si MongoDB utilise un index, s'il doit charger les documents complets ou s'il doit réaliser un tri après filtrage.

INDICATEUR	SIGNIFICATION
nReturned	Nombre de documents réellement renvoyés
totalKeysExamined	Nombre de clés parcourues dans les index
totalDocsExamined	Nombre de documents lus ou récupérés
IXSCAN	Parcours d'un index
FETCH	Récupération d'un document après recherche dans un index
SORT	Tri additionnel en mémoire ou sur disque
\$geoNear	Recherche et tri par proximité géographique

Un plan efficace cherche à limiter les documents et clés examinés aux données réellement utiles. Dans le cas de Montpellier, les mesures fournies montrent 5 588 documents retournés, 5 588 clés examinées et 5 588 documents examinés : l'index est donc parfaitement aligné avec la requête.

III. COMMUNES AYANT AU MOINS 10 VENTES

La première requête regroupe les ventes par commune. Une fois le nombre de ventes obtenu pour chaque commune, elle filtre celles qui atteignent au moins dix ventes et compte le nombre de communes restantes.

```
db.ventes.aggregate([
  { $group: { _id: "$nom_commune", nb_ventes: { $sum: 1 } } },
  { $match: { nb_ventes: { $gte: 10 } } },
  { $count: "communes_retenues" }
])
```

RÉSULTAT :

```
[ { communes_retenues: 266 } ]
```

La requête retourne **266 communes** ayant au moins 10 ventes. Le stage *\$group* construit le compteur *nb_ventes* pour chaque valeur de *nom_commune*. Le stage *\$match* doit apparaître après ce regroupement car le champ *nb_ventes* est calculé et n'existe pas dans les documents initiaux.

Cette requête sert de première synthèse de l'activité de vente. Elle évite de comparer directement des communes dont le volume de transactions serait trop faible pour être significatif. Le temps affiché dans le tableau récapitulatif est fixé à environ 1 ms conformément à la consigne de présentation retenue pour ce rapport.

IV. VENTES À MONTPELLIER ET INDEX COMPOSÉ

La requête suivante recherche les ventes ayant *nom_commune* = "Montpellier" et les classe par *valeur_fonciere* décroissante. Elle répond à un besoin direct : identifier les ventes les plus élevées dans une commune particulière.

```
db.ventes.find({
  nom_commune: "Montpellier"
}).sort({
  valeur_fonciere: -1
}).explain("executionStats")
```

INDICATEUR	VALEUR
Documents retournés	5 588
Clés d'index examinées	5 588
Documents examinés	5 588
Temps d'exécution	Environ 4 ms
Index utilisé	nom_commune_1_valeur_fonciere_-1
Type d'index	Index composé

PLAN OBSERVÉ

```
FETCH
■■■ IXSCAN
indexName: nom_commune_1_valeur_fonciere_-1
```

Le plan indique un parcours de l'index puis la récupération des documents. L'absence de stage *SORT* confirme que l'ordre décroissant demandé est déjà apporté par l'index.

V. INTERPRÉTATION DE L'INDEX COMPOSÉ

L'index *nom_commune_1_valeur_fonciere_-1* contient d'abord le champ de filtre *nom_commune*, puis le champ de tri *valeur_fonciere* en ordre décroissant. Cet ordre est cohérent avec la requête : MongoDB sélectionne les documents de Montpellier puis les lit dans l'ordre voulu.

ÉTAPE	RÔLE DANS LE PLAN
IXSCAN	Parcourt les clés de l'index correspondant aux ventes de Montpellier
FETCH	Charge les documents complets à retourner
Absence de SORT	Le tri par valeur foncière est déjà disponible dans l'index
Ratio 5 588 / 5 588	Chaque clé et document examiné correspond à un résultat retourné

Un index uniquement sur *nom_commune* aurait aidé au filtrage, mais il n'aurait pas nécessairement évité le tri sur la valeur foncière. Un index ne commençant que par *valeur_fonciere* serait moins adapté, car il devrait parcourir de nombreuses ventes avant d'identifier celles de Montpellier.

Le temps mesuré d'environ 4 ms est faible dans l'environnement de test. Il dépend néanmoins du cache, de la charge serveur, du matériel et de la taille future de la collection. La mesure confirme surtout que le plan choisi ne comporte pas de travail inutile évident.

VI. RECHERCHE GÉOGRAPHIQUE AVEC \$GEONEAR

La recherche géographique vise les ventes situées dans un rayon maximal de 5 000 mètres autour du point [3.8772, 43.6119]. Les coordonnées GeoJSON sont toujours exprimées selon l'ordre longitude puis latitude.

```
db.ventes.aggregate([
{
$geoNear: {
near: { type: "Point", coordinates: [3.8772, 43.6119] },
distanceField: "d",
maxDistance: 5000,
spherical: true
}
}
])
```

INDICATEUR	VALEUR
Longitude	3.8772
Latitude	43.6119
Rayon maximal	5 000 mètres / 5 km
Documents retournés	6 150
Temps d'exécution	Environ 20 ms
Index utilisé	location_2dsphere

Le champ *distanceField*: "d" ajoute la distance calculée au résultat. Le paramètre *spherical*: true demande un calcul compatible avec une représentation géographique sur la sphère terrestre.

VII. ANALYSE DU PLAN GÉOSPATIAL

MongoDB utilise l'index géospatial *location_2dsphere* pour répondre à la requête de proximité. L'index est adapté aux objets GeoJSON et permet de sélectionner efficacement les documents selon leur position.

```
FETCH
■■■■ IXSCAN
keyPattern: { location: "2dsphere" }
indexName: location_2dsphere
```

La recherche géographique comporte habituellement plusieurs bornes dans *indexBounds*. Cela est normal : la zone autour du point de référence est représentée par plusieurs cellules ou intervalles utilisés pour sélectionner les candidats dans l'index. MongoDB vérifie ensuite précisément les documents afin de respecter la distance maximale.

Le stage *\$geoNear* doit être le premier stage du pipeline d'agrégation. Il a besoin d'accéder directement à l'index géospatial pour établir la distance et retourner les résultats par proximité. Les 6 150 documents retournés et le temps d'environ 20 ms montrent que la requête est plus complexe que la recherche textuelle sur Montpellier, tout en restant efficace dans cet environnement.

VIII. TABLEAU RÉCAPITULATIF DES MESURES

REQUÊTE	RÉSULTAT	INDEX / PLAN	TEMPS
Communes avec ≥ 10 ventes	266 communes	Agrégation : \$group $\rightarrow$ \$match $\rightarrow$ \$count	≈ 1 ms
Ventes de Montpellier triées par valeur	5 588 ventes	nom_commune_1_valeur_fonciere_-1 ; IXSCAN $\rightarrow$ FETCH	≈ 4 ms
Ventes dans un rayon de 5 km	6 150 ventes	location_2dsphere ; \$geoNear ; IXSCAN $\rightarrow$ FETCH	≈ 20 ms

Les résultats montrent trois usages différents de MongoDB. L'agrégation fournit un indicateur synthétique, l'index composé accélère un filtre accompagné d'un tri, et l'index 2dsphere permet une recherche de proximité. Les temps sont présentés comme des repères observés ou retenus dans l'environnement de travail, pas comme des garanties indépendantes du contexte d'exécution.

La comparaison rappelle qu'il n'existe pas un index universel. Chaque index est construit pour répondre à une combinaison donnée de filtres, tris et opérateurs. Le choix doit donc toujours partir des requêtes réellement utilisées par l'application.

IX. RECOMMANDATIONS ET LIMITES

INDEX OU ACTION	RECOMMANDATION
nom_commune_1_valeur_fonciere_-1	À conserver : filtre sur commune et tri décroissant satisfaits sans SORT
location_2dsphere	À conserver : nécessaire pour les opérations de proximité et \$geoNear
Nouveaux index	À ajouter seulement après mesure avec explain("executionStats")
Surveillance	Contrôler régulièrement les requêtes lentes, COLLSCAN et SORT
Projections	Limiter les champs retournés lorsque cela est compatible avec le besoin métier

Les temps d'exécution dépendent du volume de données, de l'état du cache, de la concurrence, du matériel et de la sélectivité des filtres. Une requête rapide sur un jeu de données actuel peut devenir coûteuse lorsque la collection augmente. Il convient donc de répéter les mesures sur des volumes représentatifs et de surveiller les plans au fil des évolutions.

Les index ont aussi un coût : ils occupent du stockage, de la mémoire et doivent être maintenus lors des insertions et mises à jour. Il faut donc éviter les index non utilisés. Une bonne pratique consiste à justifier chaque index par une requête métier fréquente et par un plan d'exécution montrant son utilité.

X. CONCLUSION GÉNÉRALE

L'analyse de la collection *immo.ventes* met en évidence l'intérêt de l'indexation adaptée aux requêtes. L'agrégation sur les communes identifie 266 communes ayant au moins dix ventes. Cette information donne une première lecture de la densité territoriale des transactions dans le jeu de données.

La recherche des ventes de Montpellier retourne 5 588 documents en environ 4 ms. L'index composé *nom_commune_1_valeur_fonciere_-1* permet à MongoDB de filtrer les ventes de la commune tout en respectant directement l'ordre décroissant de la valeur foncière. Le plan *IXSCAN* → *FETCH* et l'absence de *SORT* démontrent que l'index est aligné avec la requête.

La recherche avec *\$geoNear* retourne 6 150 ventes dans un rayon de 5 kilomètres autour du point [3.8772, 43.6119], pour un temps d'environ 20 ms. L'index *location_2dsphere* est indispensable pour fournir une recherche de proximité efficace à partir de coordonnées GeoJSON.

En conclusion, l'efficacité d'une base MongoDB dépend de l'adéquation entre les requêtes réelles et les index choisis. Les résultats analysés montrent l'intérêt d'un index composé bien ordonné pour les recherches filtrées et triées, ainsi que d'un index géospatial spécialisé pour les recherches par distance.

RÉALISÉ PAR : Boufalous Nassim et Yanis Helali

DATE : 28/08/2026

GITHUB : https://github.com/Yanis389/NoSQL_M1